

Reviewer Pack — Minimal Local Coherence and DPLL Proof Path Length Lower Bou...

Entry b743b8be2ac9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 07:50:21 UTC

Minimal Local Coherence and DPLL Proof Path Length Lower Bound

Entry ID: b743b8be2ac9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 07:50:21 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Local Coherence) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Path Length)

Statement:

For every Boolean satisfiability instance with n variables, the local coherence of the associated category of clauses is $\Omega(n^{2/3})$. Equivalently: if a category C has local coherence less than $\Omega(n^{2/3})$, then there exists a DPLL proof for the instance with path length $O(n^{1/3})$.

Rationale (proposer's reasoning):

Local coherence provides a categorical invariant that captures the complexity of interactions between objects in a category. It is believed to have potential as a complexity measure due to its connection with algebraic properties. A lower bound on local coherence would imply a lower bound on proof path length, potentially linking categorical concepts with computational complexity.

Taxonomy category: Category Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bcb0584c0f8bd0d4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across at least 24 out of 30 seeds, all clauses' local coherence is $\geq \Omega(n^{2/3})$ AND any instance with local coherence $< \Omega(n^{2/3})$ has a DPLL proof path length $\leq O(n^{1/3})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - category theory AND local coherence AND boolean satisfiability AND DPLL proof path length - local coherence in category theory AND boolean satisfiability problem AND DPLL lower bound - DPLL proof path length lower bound related to category theory of clauses

Top relevant hits considered: - [http://arxiv.org/abs/1606.01722v2] A Constructive Proof of Coherence for Symmetric Monoidal Categories Using Rewriting - [http://arxiv.org/abs/2410.21524v2] Locally rigid ∞ -categories - [http://arxiv.org/abs/1005.0209v2] Approximations and adjoints in homotopy categories - [http://arxiv.org/abs/1103.5740v2] Generating and Searching Families of FFT Algorithms - [http://arxiv.org/abs/2404.02414v2] A simple lower bound for the complexity of estimating partition functions on a quantum computer - [http://arxiv.org/abs/1801.08709v2] Adaptive Lower Bound for Testing Monotonicity on the Line - [http://arxiv.org/abs/1912.10642v7] Notes on Category Theory with examples from basic mathematics - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_instance(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([True, False]) for _ in range(n)]
            if any(clause[i] != clause[j] for i, j in combinations(range(n), 2)):
                clauses.append(clause)
        return clauses

    def local_coherence(clauses):
        n = len(clauses[0])
        coherence = 0
        for i in range(n):
            for j in range(i + 1, n):
                count = sum(1 for clause in clauses if clause[i] != clause[j])
                coherence += count / (n - 1)
        return coherence / n
```

```

def dpll(clauses, assignment=[]):
    if not clauses:
        return True
    literal = next(lit for lit in range(-n, n + 1) if lit not in assignment and -lit not in as
    if literal > 0:
        assignment.append(literal)
    else:
        assignment.append(-literal)
    if dpll(clauses, assignment):
        return True
    assignment.pop()
    if literal > 0:
        assignment.append(-literal)
    else:
        assignment.append(literal)
    if dpll(clauses, assignment):
        return True
    assignment.pop()
    return False

def combinations(iterable, r):
    pool = list(iterable)
    n = len(pool)
    if r > n:
        return
    indices = list(range(r))
    yield tuple(pool[i] for i in indices)
    while True:
        for i in reversed(range(r)):
            if indices[i] != i + n - r:
                break
        else:
            return
        indices[i] += 1
        for j in range(i + 1, r):
            indices[j] = indices[j - 1] + 1
        yield tuple(pool[i] for i in indices)

def path_length(clauses, assignment=[]):
    if not clauses:
        return 0
    literal = next(lit for lit in range(-n, n + 1) if lit not in assignment and -lit not in as
    if literal > 0:
        assignment.append(literal)
    else:
        assignment.append(-literal)
    length = path_length(clauses, assignment)
    assignment.pop()
    if literal > 0:
        assignment.append(-literal)
    else:
        assignment.append(literal)
    assignment.pop()
    return length + 1

n_values = [5, 10, 15, 20, 30, 40]

```

```

results = []

for n in n_values:
    instances_tested = 0
    n_max = n
    conjecture_holds = True
    counterexample = ""

    for _ in range(5): # Sample 5 random instances per size
        clauses = generate_instance(n)
        coherence = local_coherence(clauses)
        if coherence < n**(2/3):
            path_len = path_length(clauses)
            if path_len > n**(1/3):
                conjecture_holds = False
                counterexample = f"n={n}, coherence={coherence}, path_length={path_len}"
                break
            instances_tested += 1

    results.append({
        "metric_name": "local_coherence",
        "metric_value": coherence,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    })

return {
    "seed": seed,
    **results[0],
    "mean_metric_value": sum(result["metric_value"] for result in results) / len(results),
    "std_metric_value": math.sqrt(sum((result["metric_value"] - results[0]["mean_metric_value"])**2 for result in results) / len(results)),
    "support_fraction": sum(1 for result in results if result["conjecture_holds"]) / len(results)
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["mean_metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["mean_metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["support_fraction"] >= 0.8) / len(results)

    if all(result["support_fraction"] >= 0.8 for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:

```

```
print("RESULT: INCONCLUSIVE insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26513
2	propose	ollama_remote	glm4:latest	0	0	14658
3	propose	ollama_remote	glm4:latest	0	0	17804
4	preregistration	ollama_remote	glm4:latest	0	0	9415
5	novelty	ollama_remote	glm4:latest	0	0	14591
6	novelty	ollama_remote	glm4:latest	0	0	10182
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44292
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13205
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8257
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14294
11	judge	ollama_remote	glm4:latest	0	0	24041

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 197252 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/b743b8be2ac9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b743b8be2ac9.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b743b8be2ac9.tar.gz` (if generated)